



DATABASE · SQL · ANALYTICS

SQL Project on Online Book Store

A PostgreSQL database built from three CSV exports — with table creation, data import, and 20 real business queries.

Books_data.csv · Customers_data.csv · Orders_data.csv



PROJECT OVERVIEW

From Raw CSVs to Business Insight

This project builds a PostgreSQL database for an online book store from three linked CSV exports, then uses SQL — from table creation through advanced joins — to answer the questions a real store manager would ask.

- Create Books, Customers and Orders tables with primary & foreign keys
- Import each CSV with PostgreSQL's COPY command
- Answer 11 foundational (basic) business questions
- Answer 9 analytical (advanced) questions using joins, grouping and aggregation

3

Linked Tables

20

SQL Queries

2

Skill Levels

PROJECT GOAL

Turn a normalized book-store schema into decision-ready answers: best sellers, top customers, remaining stock and revenue — built end-to-end in PostgreSQL.

Database Schema & Relationships

Three CSV files become three tables, joined through shared key columns of matching name and type.

 BOOKS	
Book_ID	
Title	
Author	
Genre	
Published_Year	
Price	
Stock	

 CUSTOMERS	
Customer_ID	
Name	
Email	
Phone	
City	
Country	

 ORDERS	
Order_ID	
Customer_ID	
Book_ID	
Order_Date	
Quantity	
Total_Amount	

• Book_ID links Books ↔ Orders

• Customer_ID links Customers ↔ Orders

Creating Tables & Loading Data

Each table is created with PostgreSQL DDL, including primary and foreign keys, then populated from its CSV with COPY.

BOOKS

```
CREATE TABLE Books (  
  Book_ID SERIAL PRIMARY KEY,  
  Title VARCHAR(100),  
  Author VARCHAR(100),  
  Genre VARCHAR(50),  
  Published_Year INT,  
  Price NUMERIC(10,2),  
  Stock INT  
);
```

CUSTOMERS

```
CREATE TABLE Customers (  
  Customer_ID SERIAL PRIMARY KEY,  
  Name VARCHAR(100),  
  Email VARCHAR(100),  
  Phone VARCHAR(15),  
  City VARCHAR(50),  
  Country VARCHAR(150)  
);
```

ORDERS

```
CREATE TABLE Orders (  
  Order_ID SERIAL PRIMARY KEY,  
  Customer_ID INT REFERENCES  
    Customers(Customer_ID),  
  Book_ID INT REFERENCES  
    Books(Book_ID),  
  Order_Date DATE,  
  Quantity INT,  
  Total_Amount NUMERIC(10,2)  
);
```

```
COPY <table> FROM '<table>_data.csv' DELIMITER ',' CSV HEADER;
```

run once per table — Books, Customers, Orders

Basic Queries — The Questions

Foundational lookups and filters across single tables.

- 1 Retrieve all books in the “Fiction” genre
- 2 Find books published after the year 1950
- 3 List all customers from Canada
- 4 Show orders placed in November 2023
- 5 Retrieve the total stock of books available
- 6 Find the details of the most expensive book
- 7 Show customers who ordered more than 1 quantity of a book
- 8 Retrieve all orders where the total amount exceeds \$20
- 9 List all genres available in the Books table
- 10 Find the book with the lowest stock
- 11 Calculate the total revenue generated from all orders

Queries 1–3

1 Books in the “Fiction” genre

```
SELECT *  
FROM Books  
WHERE Genre = 'Fiction';
```

Note: genre match is case-sensitive

2 Books published after 1950

```
SELECT *  
FROM Books  
WHERE Published_Year > 1950;
```

3 Customers located in Canada

```
SELECT *  
FROM Customers  
WHERE Country = 'Canada';
```

Result: 3 customers

Queries 4–6

4 Orders placed in November 2023

```
SELECT *  
FROM Orders  
WHERE Order_Date BETWEEN '2023-11-01' AND '2023-11-30';
```

5 Total stock of books available

```
SELECT SUM(Stock) AS Total_Stock  
FROM Books;
```

Result: 25,056 books

6 Details of the most expensive book

```
SELECT *  
FROM Books  
ORDER BY Price DESC LIMIT 1;
```

Top result: "Proactive System-Worthy Orchestration"

Queries 7–9

7 Customers who ordered more than 1 quantity

```
-- Generalized answer
SELECT *
FROM Orders
WHERE Quantity > 1;

-- With customer name (specific columns)
SELECT C.Name, O.Order_ID, O.Quantity
FROM Orders O
JOIN Customers C ON O.Customer_ID = C.Customer_ID
WHERE Quantity > 1
ORDER BY Quantity DESC;
```

8 Orders where total amount exceeds \$20

```
SELECT *
FROM Orders
WHERE Total_Amount > 20;
```

9 All genres available in the Books table

```
SELECT DISTINCT Genre
FROM Books;
```

Queries 10–11

10 Book with the lowest stock

```
SELECT *  
FROM Books  
ORDER BY Stock LIMIT 1;
```

Result: "Networked Systemic Implementation"

11 Total revenue generated from all orders

```
SELECT SUM(Total_Amount) AS Total_Revenue  
FROM Orders;
```

PART TWO

Advanced Queries — The Questions

Multi-table joins, grouping and aggregation for deeper business insight.

- 1 Total number of books sold for each genre
- 2 Average price of books in the “Fantasy” genre
- 3 Customers who have placed at least 2 orders
- 4 The most frequently ordered book
- 5 Top 3 most expensive books in the ‘Fantasy’ genre
- 6 Total quantity of books sold by each author
- 7 Cities where customers who spent over \$30 are located
- 8 The customer who spent the most on orders
- 9 Stock remaining after fulfilling all orders

Queries 1–3

1 Total number of books sold, per genre

```
SELECT B.Genre, SUM(O.Quantity) AS Total_Books_Sold
FROM Orders O
JOIN Books B ON O.Book_ID = B.Book_ID
GROUP BY B.Genre;
```

2 Average price of books in the “Fantasy” genre

```
SELECT ROUND(AVG(Price), 2) AS Average_Price
FROM Books
WHERE Genre = 'Fantasy';
```

3 Customers who have placed at least 2 orders

```
SELECT O.Customer_ID, C.Name, COUNT(O.Order_ID) AS Order_Count
FROM Orders O
JOIN Customers C ON O.Customer_ID = C.Customer_ID
GROUP BY O.Customer_ID, C.Name
HAVING COUNT(O.Order_ID) >= 2
ORDER BY Customer_ID;
```

Queries 4–6

4 The most frequently ordered book

```
SELECT O.Book_ID, B.Title, COUNT(O.Order_ID) AS Order_Count
FROM Orders O
JOIN Books B ON O.Book_ID = B.Book_ID
GROUP BY O.Book_ID, B.Title
ORDER BY Order_Count DESC LIMIT 1;
```

Top result: "Robust Tangible Hardware"

5 Top 3 most expensive books in the 'Fantasy' genre

```
SELECT *
FROM Books
WHERE Genre = 'Fantasy'
ORDER BY Price DESC LIMIT 3;
```

6 Total quantity of books sold, per author

```
SELECT b.Author, SUM(o.Quantity) AS Total_Books_Sold
FROM Orders o
JOIN Books b ON o.Book_ID = b.Book_ID
GROUP BY b.Author
ORDER BY Total_Books_Sold DESC;
```

Queries 7–8

7 Cities of customers who spent over \$30

```
SELECT DISTINCT c.City, Total_Amount AS Amount_Spent
FROM Orders o
JOIN Customers c ON o.Customer_ID = c.Customer_ID
WHERE o.Total_Amount > 30
ORDER BY Total_Amount DESC;
```

8 The customer who spent the most overall

```
SELECT C.Customer_ID, C.Name, SUM(O.Total_Amount) AS Total_Spent
FROM Orders O
JOIN Customers C ON O.Customer_ID = C.Customer_ID
GROUP BY C.Customer_ID, C.Name
ORDER BY Total_Spent DESC LIMIT 1;
```

Result: Kim Turner

Query 9 — Two Ways to the Same Answer

Stock remaining after fulfilling all orders

1 Method 1 — remaining stock only

```
SELECT b.Book_ID, b.Title,  
       b.Stock - COALESCE(SUM(o.Quantity), 0) AS Remaining_Stock  
FROM Books b  
LEFT JOIN Orders o ON b.Book_ID = o.Book_ID  
GROUP BY b.Book_ID;
```

2 Method 2 — shows ordered quantity too

```
SELECT B.Book_ID, B.Title, B.Stock,  
       COALESCE(SUM(O.Quantity), 0) AS Ordered_Quantity,  
       B.Stock - COALESCE(SUM(O.Quantity), 0) AS Remaining_Stock  
FROM Books B  
LEFT JOIN Orders O ON B.Book_ID = O.Book_ID  
GROUP BY B.Book_ID  
ORDER BY B.Book_ID;
```

LEFT JOIN + COALESCE keeps books with zero orders

TAKEAWAYS

SQL Skills Demonstrated



Schema Design (DDL)

CREATE TABLE with primary & foreign keys



Data Loading

Bulk import with PostgreSQL COPY ... CSV HEADER



Filtering & Sorting

WHERE, ORDER BY, LIMIT, BETWEEN, DISTINCT



Aggregate Functions

SUM, AVG, COUNT, ROUND applied across tables



Grouping & Having

GROUP BY with HAVING for multi-row conditions



Table Joins

INNER and LEFT JOIN across Books, Customers, Orders

THANK YOU

Questions & Discussion

SQL Project · Online Book Store — Books_data.csv · Customers_data.csv · Orders_data.csv